

Deployment Runbook — Xolvon Backend

Step-by-step operating procedure for promoting this backend to staging and production, including migrations, rollback, first-admin bootstrap, and the (currently open) monitoring decision. Read [``production-operations.md``] (`production-operations.md`) for policy and [``qa-results.md``] (`qa-results.md`) for the verified state.

0. Current honest status

- **Local**: fully working — SQLite (``DB_DRIVER=sqlite``) + local-test storage (``STORAGE_DRIVER=local-test``), no cloud credentials.
- **Staging/production**: **NOT** deployed and **NOT** verified. The D1/R2 live path is blocked for lack of a deployment target, secret stores, wrangler access, and final HTTPS domains (recorded in [``qa-results.md``] (`qa-results.md`)). Nothing in this runbook may be marked done without real evidence; do not claim success from a local dry run.
- **Unblock requirements** (from ``qa-results.md``): separate staging/production secret stores with valid D1/R2/JWT values, an authenticated deployment provider/target for the Node service, pre-migration backups, log access, and real HTTPS domains for both environments.

1. Deployment architecture (documented deviation)

The backend runs as a **Node/Express NestJS service** (``@nestjs/platform-express``, ``node dist/main``). Cloudflare D1 is reached **over** its REST API through the ``D1RestAdapter`` behind the ``DatabaseService`` token (``DB_DRIVER=d1``), and R2 through the S3-compatible SDK behind ``STORAGE_PORT`` (``STORAGE_DRIVER=r2``).

HANDBOOK_BACKEND.md §8 Langkah 6/8 shows swapping ``DatabaseService`` to a Workers ``env.DB`` binding and deploying via ``vinext`/`@vinext/cloudflare`` — that path belongs to the **frontend/Workers deployment**. For this backend it is a **documented deviation** (conflict K-06 in [``decision-log.md``] (`decision-log.md`)): the service is not a Workers runtime target; do not attempt to deploy ``dist/main`` with `vinext`.

2. Configuration per environment

Copy names only from the committed templates and populate values in the target's secret store — never in Git, docs, logs, or shell history:

Template	Fixed shape (non-secret)
<code>`env.staging.example`</code>	<code>`APP_ENV=staging`</code> , <code>`DB_DRIVER=d1`</code> , <code>`STORAGE_DRIVER=r2`</code> , <code>`TRUST_PROXY_HOPS=1`</code>
<code>`env.production.example`</code>	<code>`APP_ENV=production`</code> , <code>`DB_DRIVER=d1`</code> , <code>`STORAGE_DRIVER=r2`</code> , <code>`TRUST_PROXY_HOPS=1`</code>

Both templates still need real values for ``CLOUDFLARE_ACCOUNT_ID``, ``CLOUDFLARE_D1_DATABASE_ID``, ``CLOUDFLARE_API_TOKEN``, ``R2_ENDPOINT``, ``R2_BUCKET``, ``R2_ACCESS_KEY_ID``, ``R2_SECRET_ACCESS_KEY``, ``JWT_SECRET``, ``FRONTEND_URL``, and optionally ``PORT``. In addition, set the two runtime-optional names per environment once domains are decided (DL-020/DL-021/DL-

027):

- `CORS\_ALLOWED\_ORIGINS` – final staging/production frontend origins (comma separated). ****OPEN** – owner has not supplied final domains. **** Wildcards are rejected when credentials mode is on.**
- `CSP\_CONNECT\_SRC` – frontend origins allowed in CSP fetch/source lists. ****OPEN**** until confirmed; unset is safe (defaults to `self` + `R2\_ENDPOINT` origin).

Validation is fail-fast at bootstrap (`src/config/env.validation.ts`):
`dl` requires the three `CLOUDFLARE\_\*` names; `r2` requires the four `R2\_\*` names; `JWT\_SECRET` ≥ 32 chars and a valid `FRONTEND\_URL` are always required. `TRUST\_PROXY\_HOPS` must be a non-negative integer string or the process aborts before binding a port.

3. Provisioning D1 and R2 (once, per environment)

1. Auth wrangler on the operator machine: `npx wrangler login`.
2. D1 database – staging already exists and is the committed binding target in `wrangler.jsonc` (`database\_name: "xolvon-staging"`, binding `DB`, `migrations\_dir: "src/database/migrations"`). Production requires a new `npx wrangler d1 create xolvon-production` (name TBD by owner); record the resulting id ****only**** in the secret store as `CLOUDFLARE\_D1\_DATABASE\_ID` – never in documents.
3. R2 bucket: `npx wrangler r2 bucket create <bucket>`; mint an API token scoped to read/write on that bucket; store endpoint/keys as the four `R2\_\*` env names.
4. Deployment target for the Node process: ****not yet provisioned**** – any Node-capable host/PaaS that can hold a secret store and sit behind HTTPS with a known proxy hop count. Decide and record it in [`decision-log.md`](decision-log.md) before first deploy (relates to PRD \$91.4, open).

4. Migration apply

- ****Staging/production (D1):****
`npx wrangler d1 migrations apply xolvon-staging --remote` (repeat with the production database name for production). Take a D1 backup before the first apply of any release.
- The in-repo runner (`src/database/migrate.ts`) applies files in `src/database/migrations/` in filename order, records each with a SHA-256 hash in `\_migrations`, skips applied files, and ****throws on hash drift**** – applied migrations are immutable. The HTTP server does not auto-migrate.
- Never edit an applied migration; add a new numbered file (see [`README.md`](README.md) §5).

5. Staging → production deployment procedure

Do staging first; production is the same sequence with production values.

5.1 Staging

1. Prereqs from §0 satisfied; backup of the staging D1 taken.
2. Apply pending migrations (§4) against the staging database.
3. Deploy the built artifact (`npm ci && npm run build` → ship `dist/`, or the provider's equivalent) with the staging env populated (§2).
`package.json` exposes only `deploy: "nest deploy"` as a placeholder – it

- targets no configured staging/production platform, so the actual deployment mechanism is the provider target from §3.4 (still unprovisioned).
4. Health check: ``curl -i https://<staging-host>/api`` → expect 200. Verify headers on the response: CSP present with the configured origins, ``X-Content-Type-Options: nosniff``, ``X-Frame-Options: DENY``, ``Referrer-Policy: strict-origin-when-cross-origin``, no ``X-Powered-By``; ****no HSTS on staging**** (production-only, by design).
 5. Functional smoke (clean accounts, throwaway data):
 - ``POST /api/auth/register`` then ``login``; repeat login 6× → 6th returns 429 (auth 5/min).
 - ``GET /api/courses`` → 200; burst 31× → 429 (catalog 30/min).
 - Browser-mutation integrity: ``POST /api/orders`` with a disallowed ``Origin:`` header → 403; without browser headers + valid Bearer → passes auth path.
 - CORS: allowed origin ``OPTIONS`` preflight returns ``Access-Control-Allow-Origin`` + credentials; unknown origin does not.
 6. First admin (§6).
 7. Record evidence (status codes, header dumps) without copying tokens or secret values anywhere.

5.2 Production

1. Same as staging with production secret store, plus:
2. ``APP_ENV=production`` → HSTS ``max-age=31536000``; ``includeSubDomains`` must appear; confirm only after HTTPS is terminating correctly, since HSTS is sticky in browsers.
3. ``CORS_ALLOWED_ORIGINS`` / ``CSP_CONNECT_SRC`` set to the final owner-approved domains (blocked on DL-027 – OPEN).
4. Repeat the §5.1 smoke matrix against the production host on a clean account, then promote real content.
5. Any production success claim requires captured live evidence – the current recorded state is ****BLOCKED**** (`[`qa-results.md`](qa-results.md)`).

6. First-admin bootstrap (controlled)

There is no public admin registration route and no role input is ever accepted from a client (DL-005/DL-017). Seed the first admin from a controlled operator shell with the target environment's secret store attached, using the variable names only (values from the secret store; avoid shell history – e.g. read them from a file descriptor or interactive prompt):

```
```bash
ADMIN_BOOTSTRAP_EMAIL='<admin email>' \
ADMIN_BOOTSTRAP_PASSWORD='<from secret store>' \
npm run seed:admin
```
```

Behavior (``src/database/seed-admin.ts`` → ``admin-bootstrap.ts``): applies pending migrations, hashes the password with Argon2id via ``PasswordService``, inserts ``role='admin'``, ``status='active'``, prints only ``created`` or ``already_exists``, is idempotent for the same admin email, and ****refuses**** (conflict error) to touch an existing non-admin account.

7. Rollback

Authoritative procedure: `[`production-operations.md`](production-operations.md)` "Rollback procedure". Short form, in order:

1. Stop promotion; record deployment id, last-applied migration filename, and incident timestamp (no secret values).
2. ****Revert the deploy**** to the last known-good immutable build and confirm it boots with the target secret store (``GET /api` health`).
3. ****Revert the last migration**** only if the failed release applied one: restore the pre-migration database backup/snapshot taken before that apply. The runner is forward-only and creates no down-migrations.
4. If the platform cannot restore, ship a ****compensating migration**** as a new numbered file that reverses the affected change. ****Never edit an applied migration**** – ``_migrations`` hash drift must fail closed.
5. Re-run the migration set against a disposable local copy first; verify ``PRAGMA foreign_key_check`` and the affected flows; redeploy known-good.
6. Keep the failed release isolated until root cause + forward fix are reviewed.

8. Monitoring & alerting – OPEN DECISION

``MonitoringPort`` (``src/observability/monitoring.port.ts``: ``captureError``, ``recordEvent``) is ****interface-only**** – no provider SDK, endpoint, recipients, or credential is wired (DL-022). Log/monitor hygiene in the meantime (enforced by ``AllExceptionsFilter`` and code review): event names, route/status context, non-sensitive identifiers only; never passwords, tokens, hashes, signed URLs, credentials, or echoed untrusted input.

Before any production monitoring wiring, a new decision entry must record provider, alert recipients, data boundary, and secret-store location. Until then, treat "monitoring" as: structured stdout logs + provider platform metrics + the §5.1/§5.2 smoke checks per release.

9. Post-deploy checklist (per release)

- [] Migrations applied cleanly (count matches expected new files; no drift)
- [] Health ``GET /api`` 200 through HTTPS
- [] Header matrix correct for the env (HSTS only in production)
- [] CORS allow/deny spot check; no wildcard-with-credentials
- [] Auth 5/min + one named group 429 observed
- [] Disallowed-Origin browser mutation → 403
- [] Admin seed/verify/activate/revoke happy path on clean accounts
- [] Evidence captured with zero secret values
- [] ``qa-results.md`` / runbook updated with real status (or an honest BLOCKED)